

Engenharia de Software 2

(ES)

Universidade Federal do Piauí

Prof. Leonardo Vasconcelos
leonardo.vasconcelos@ufpi.edu.br

Na aula anterior ...

- Apresentação do Curso
- Trabalho 17/03

Trabalho

Trabalho

- Somente 1 pessoa irá entregar por grupo
- Grupo de 1, 2 ou 3 pessoas (**sem brigas no final**)
 - Se alguém do grupo faltar à apresentação, o restante do grupo será responsável por cobrir a ausência.
 - A pessoa que faltar, irá apresentar o trabalho sozinho
 - Será descontado 1 ponto por reclamação de quem vier reclamar do grupo sem uma solução
 - Trabalhos individuais não serão aceitos, salvo em casos excepcionais.



Trabalho

- Separar em grupos de 1, 2 ou 3 pessoas
- Escolher um problema que desejam resolver durante o curso (para a criação de uma startup)
- Não focar em soluções, focar no problema
- Resumir o problema em uma pergunta
- Apresentar em 5 minutos o problema em sala de aula no dia 17/03.

Aviso

Aviso

- Palestra on-line Pesquisa Ação
- Denise Felippo
- 26/03/2026

© COPYRIGHT

- Esta parte da apresentação é baseada nos slides previamente elaborados pelo professor Leonardo Murta, para a disciplina de Engenharia de Software 2, no Instituto de Computação da Universidade Federal Fluminense.



Revisão



Requisitos Funcionais

Requisitos Funcionais

- São declarações dos serviços que o sistema deve fornecer, do modo como o sistema deve reagir a determinadas entradas e de como deve se comportar em determinadas situações.

Requisitos Funcionais

- O sistema deve permitir que os usuários façam login utilizando credenciais válidas (e-mail e senha) e devem receber uma mensagem de erro em caso de falha.
- O sistema deve permitir que os usuários busquem produtos por nome, categoria ou palavras-chave na barra de pesquisa
- O sistema deve permitir que os usuários adicionem produtos ao carrinho de compras e vejam a quantidade e o valor total atualizados em tempo real.

Requisitos Funcionais

- Narrativa livre
 - “O sistema deve mostrar uma mensagem de status”
- Lista de requisitos
 - RF-1: Uma mensagem de status deve ser mostrada na área inferior da janela (desenho da Fig.1)
 - RF-2: A mensagem deve ser atualizada a cada 60 segundos, com tolerância de 10 segundos para mais ou para menos
 - RF-3: A mensagem deve estar sempre visível
 - RF-4: Se a mensagem for referente a uma tarefa em andamento, o percentual de andamento deve ser mostrado
 - RF-5: Se a mensagem for referente a uma tarefa já terminada, isso deve ser informado com o texto “Finalizada”

Exemplo

Escopo do Sistema

- O sistema para o Restaurante Universitário da Universidade Federal do Piauí (RU/UFPI) terá como objetivo automatizar e otimizar as operações do restaurante universitário. Ele será desenvolvido para atender alunos, servidores e gestores, proporcionando funcionalidades para controle de refeições, pagamentos, gestão de cardápios, relatórios e controle de estoque. O sistema também permitirá o acompanhamento das operações pelos administradores e a visualização de informações relevantes para os usuários.

Requisitos Funcionais

- Cadastro de Usuários: O sistema deve permitir que alunos e servidores se cadastrem, informando dados como nome, matrícula, e-mail e categoria (aluno ou servidor).
- Login de Usuários: O sistema deve permitir que usuários façam login com suas credenciais (matrícula ou e-mail e senha).
- Recuperação de Senha: O sistema deve permitir que usuários recuperem suas senhas via e-mail ou matrícula.
- Compra de Créditos: O sistema deve permitir que os usuários comprem créditos para pagar refeições, com diferentes métodos de pagamento (boleto, cartão de crédito ou débito).
- Consulta de Saldo: O sistema deve exibir o saldo de créditos do usuário para consulta a qualquer momento.

Requisitos Funcionais

- Visualização de Informações Nutricionais: O sistema deve exibir informações nutricionais detalhadas sobre cada refeição no cardápio.
- Gestão de Estoque: O sistema deve permitir o controle de estoque de alimentos, com alertas de baixa de produtos.
- Relatórios de Consumo: O sistema deve gerar relatórios sobre o consumo de refeições, divididos por data, categoria de usuários (aluno ou servidor) e período.
- Relatórios Financeiros: O sistema deve gerar relatórios financeiros detalhados, com transações, compra de créditos e gastos por período.
- Emissão de Comprovantes: O sistema deve emitir comprovantes eletrônicos de pagamento e reserva de refeições.
- Controle de Acesso: O sistema deve controlar o acesso ao restaurante, verificando a reserva e o saldo de créditos no momento da entrada.

Requisitos Não Funcionais

Requisitos não Funcionais

- São restrições sobre os serviços ou funções oferecidas pelo sistema.
- Sinônimo: atributos de qualidade
- Disponibilidade
 - DS-1: O sistema deve ficar disponível por 99,5% do tempo nos dias úteis, das 6h às 22h, e 99,95% do tempo nos dias não úteis, das 16h às 18h
- Eficiência
 - EF-1: Em condições de pico de uso, deve ter uma reserva de 25% de capacidade de processamento e memória
 - EF-2: O cálculo de interferência deve ser finalizado com sucesso em menos de 5 minutos
 - EF-3: O módulo de parser de XML deve ser capaz de processar 1.000.000 de documentos por segundo

Requisitos não Funcionais

- Flexibilidade
 - FL-1: Um novo tipo de sensor deve poder ser configurado no sistema em menos de 3 horas
- Integridade
 - IN-1: Transações históricas dos consumidores só poderão ser vistas por usuários com privilégios de “auditor”
- Interoperabilidade
 - IT-1: O sistema deve ser capaz de importar dados tanto do MS Office (versão 2003 ou superior) quanto do OpenOffice (versão 2.4 ou superior)
- Confiabilidade
 - CF-1: Em cada 1.000 execuções, não mais do que 2 podem apresentar falhas de software

Requisitos não Funcionais

- Robustez
 - RB-1: Se acontecer uma falha antes do usuário salvar, o sistema deve recuperar uma versão não salva com perda de conteúdo menor que 1 minuto de trabalho
- Usabilidade
 - US-1: Um usuário treinado deve ser capaz de submeter um pedido de compra em menos que 5 minutos
 - US-2: Um usuário não treinado deve ser capaz de submeter um pedido de compra em menos que 30 minutos
 - US-3: Todos os comandos de menu devem ter teclas de atalho associadas

Requisitos não Funcionais

- Manutenibilidade
 - MN-1: Todos os métodos devem ser documentados utilizando a notação Javadoc
 - MN-2: Todos os métodos devem ter até 30 linhas de código
 - MN-3: Modificações adaptativas devido a instrumentos legais devem poder ser feitas em menos de 20 horas
- Portabilidade
 - PR-1: O sistema deve poder ser executado em sistema operacional Windows e Linux, nas arquiteturas i386, AIX e SPARC

Requisitos não Funcionais

- Reusabilidade
 - RS-1: O controle de usuários deve reutilizar componentes de autenticação já utilizados no sistema de mapeador de portas
- Testabilidade
 - TS-1: o: O sistema deve permitir a fácil criação de scripts de teste automatizados para os principais fluxos de uso

Exemplo

Escopo do Sistema

- O sistema para o Restaurante Universitário da Universidade Federal do Piauí (RU/UFPI) terá como objetivo automatizar e otimizar as operações do restaurante universitário. Ele será desenvolvido para atender alunos, servidores e gestores, proporcionando funcionalidades para controle de refeições, pagamentos, gestão de cardápios, relatórios e controle de estoque. O sistema também permitirá o acompanhamento das operações pelos administradores e a visualização de informações relevantes para os usuários.

Requisitos não Funcionais

- O sistema deve proteger os dados dos usuários, utilizando criptografia para armazenar senhas e dados de pagamento.
- O sistema deve estar disponível 99,9% do tempo, com manutenção planejada fora dos horários de pico.
- O sistema deve ser escalável para atender a uma crescente demanda de usuários, principalmente durante períodos de maior acesso, como início de semestre.
- O sistema deve responder a consultas e transações em menos de 2 segundos para manter a experiência do usuário satisfatória.
- A interface do sistema deve ser intuitiva, com navegação fácil e acessível para usuários com diferentes níveis de experiência.

Orientação a Objetos

Orientação a Objetos

- Princípios
 - Abstração
 - Encapsulamento
 - Herança
 - Polimorfismo

Orientação a Objetos

- Um objeto é a representação computacional de um elemento ou processo do mundo real
- Cada objeto possui suas características e seu comportamento
- Exemplos
 - Casa
 - Pessoa
 - Mesa
 - etc

Classes

Classes

- Estruturas fundamentais da programação orientada a objetos.
- "Moldes" que definem a estrutura.
- Representam coisas da vida real

Classes

- Exemplos
 - Banco
 - Cliente, conta bancária, gerente, etc.
 - UFPI
 - Professor, aluno, disciplina

Atributos

Atributos

- São características ou propriedades que definem um objeto
- Exemplo
 - Banco
 - nome, conta, saldo
 - Aluno
 - nome, matrícula, endereço, média

Métodos

Métodos

- Ações
- Exemplo
 - Banco
 - realizaSaque, consultaSaldo
 - Aluno
 - consultaMedia

Instâncias

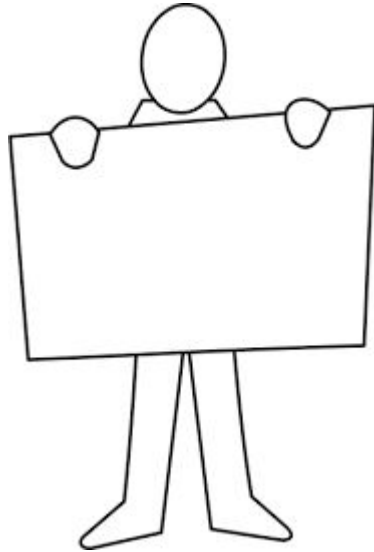
Instâncias

- É uma implementação única de uma classe
- Quando criamos um objeto a partir de uma classe, estamos instanciando essa classe
- Exemplo
 - Classe: carro
 - Instância: ferrari

Modelos

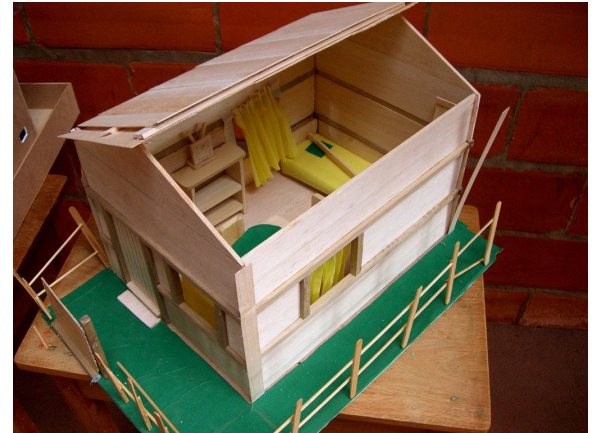
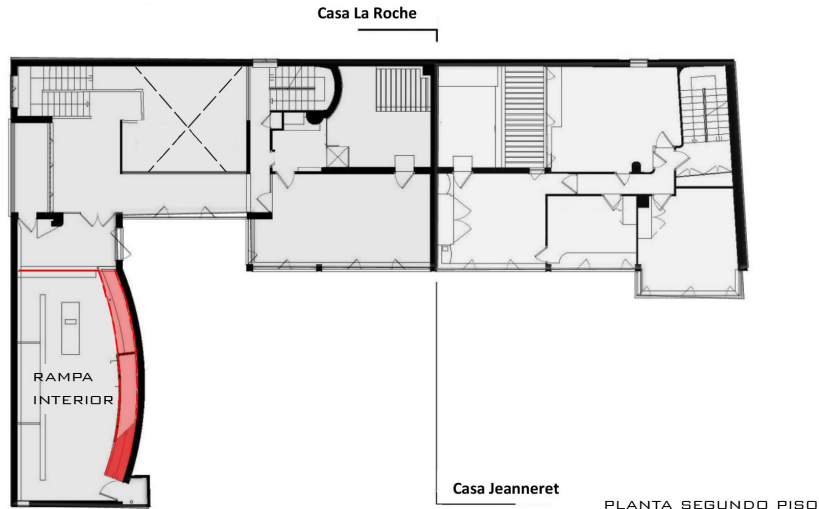
Modelos

- Os modelos são abstrações da realidade
- Eles se focam no que realmente interessa para um determinado observador em um dado momento



Modelos

- Possibilitam a comunicação entre pessoas
- Permite lidar com problemas complexos
- Testam hipóteses antes de realizá-las



Modelos

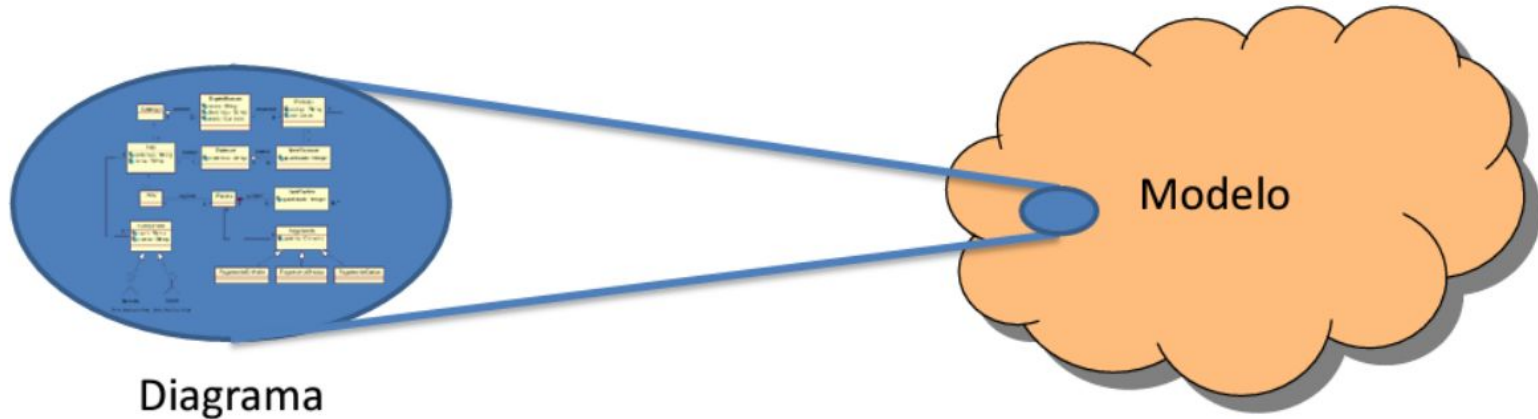
- Quais domínios do conhecimento utilizam modelos?
 - Todos!
- Modelos no domínio de desenvolvimento de software
 - Modelo também é software!
- Modelos servem para apoiar:
 - Derivação dos outros modelos
 - Codificação do sistema
- É preciso questionar a necessidade real de modelos que não servem para a derivação de outros modelos ou para a codificação do sistema!!!

Modelos x Diagramas

- O modelo contém toda a informação que representa o problema ou a solução
- O diagrama é uma visualização de parte de um modelo sob uma perspectiva
- Ou seja:
 - Se está no diagrama, está no modelo
 - Se não está no diagrama, pode ou não estar no modelo

Modelos x Diagramas

- Ou seja:
 - Se está no diagrama, está no modelo
 - Se não está no diagrama, pode ou não estar no modelo



Casos de Uso

Casos de Uso

- Atores
 - Representam as entidades que se relacionam com o sistema



Casos de Uso

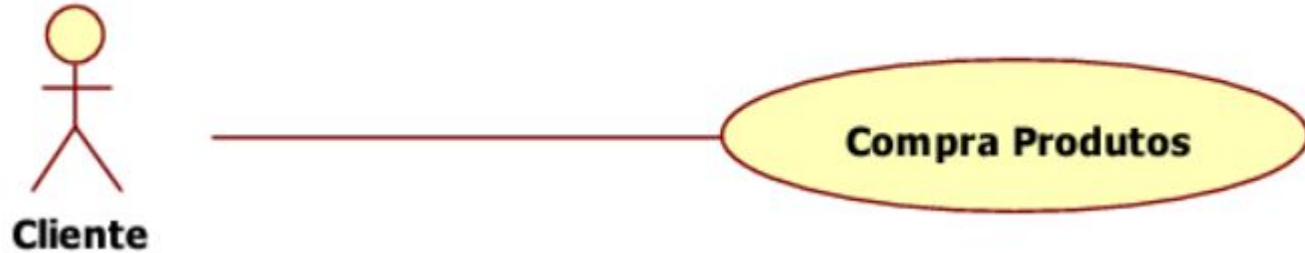
- Caso de uso



Compra Produtos

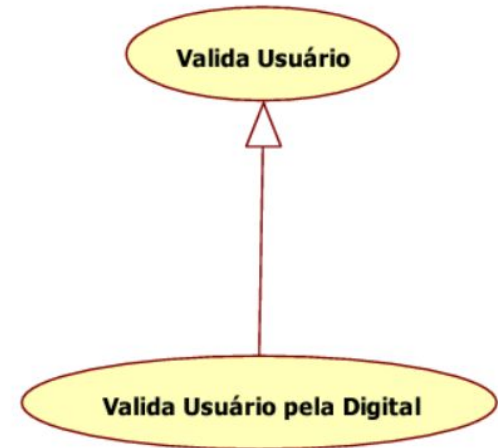
Casos de Uso

- Participação de um ator no caso de uso



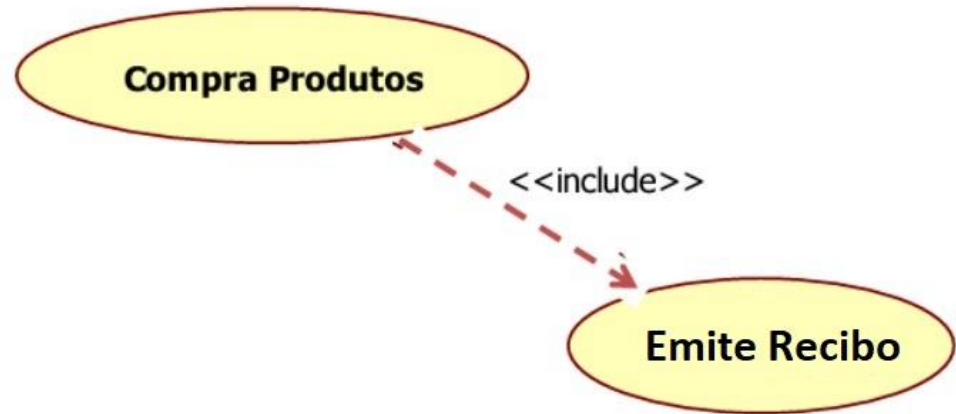
Casos de Uso

- Relação “é um” entre atores
- Relação “é um tipo de” entre casos de uso
 - Serve para representar variantes tecnológicas de um caso de uso

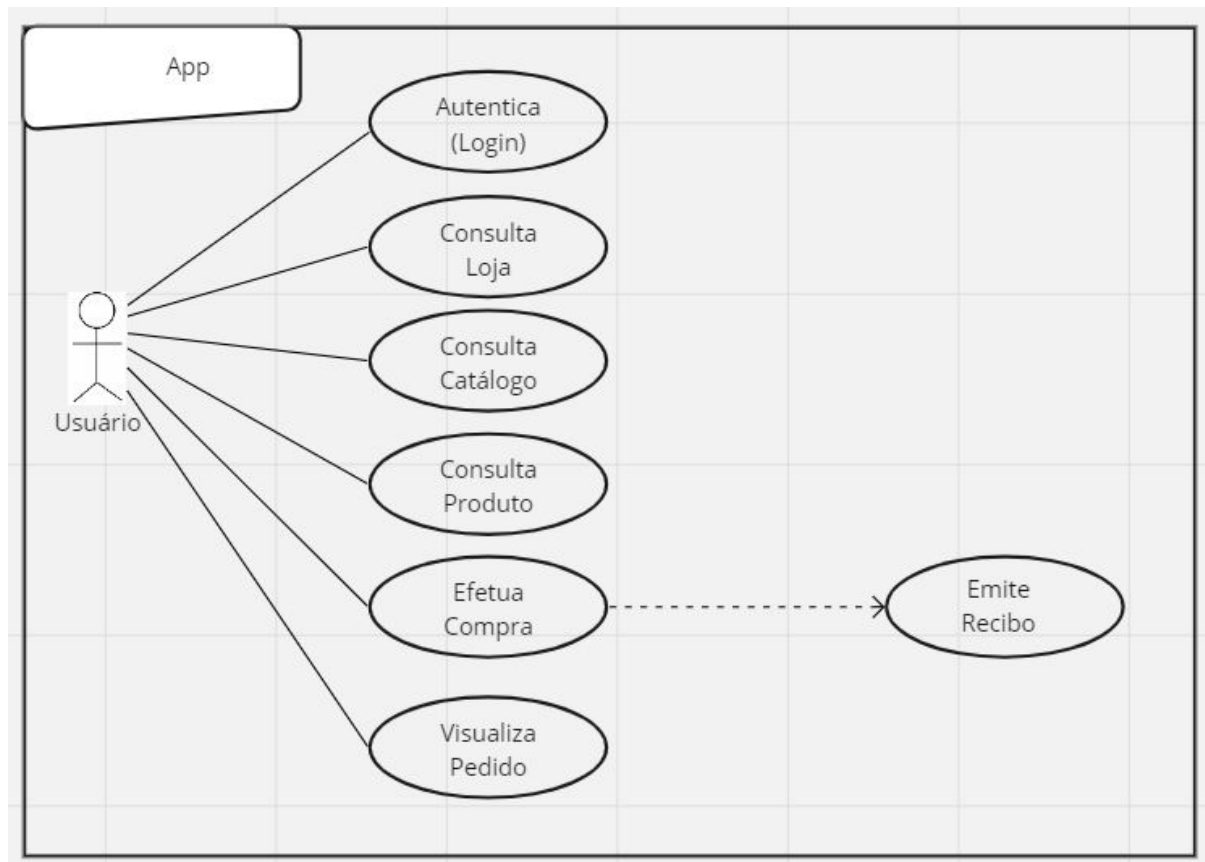


Casos de Uso - Include

- Adição de um comportamento específico em um ponto determinado do caso de uso
- Útil quando esse comportamento é repetido em diversos casos de uso do sistema



Casos de Uso - Exemplo



Casos de Uso - Extend

- É um relacionamento que mostra como um caso de uso opcional pode ser "estendido" a partir de um caso de uso base.
- Opcional



Referências

- Notas de Aula do Professor Leonardo Gresta Paulino Murta - Universidade Federal Fluminense.
- Notas de Aula dos professores Marcio Barros e Guilherme Horta para a disciplina de Engenharia de Software - CEDERJ - UFF.